

Lecture 13:

Regularization & Architectures

October 7, 2026

in-class: photometric redshifts using an MLP

ast384c/inclass/lec12_nn/photoz_mlp.ipynb

recap: we trained an MLP to predict photo-z from photometry and color

what if we feed in a galaxy image with each pixel as input?
what if we shuffle the pixels of every image in the same way?

recap: we trained an MLP to predict photo-z from photometry and color

what if we feed in a galaxy image with each pixel as input?
what if we shuffle the pixels of every image in the same way?

a neural network doesn't know what its inputs are! **but we do**

recap: we trained an MLP to predict photo-z from photometry and color

what if we feed in a galaxy image with each pixel as input?
what if we shuffle the pixels of every image in the same way?

a neural network doesn't know what its inputs are! **but we do**

Regularization & Architectures

using our **prior knowledge** to train neural networks

how do we learn the basis from data?

how do we learn the weights and biases from data?

$$\{(x_i, y_i)\}_{i=1}^N \quad \text{“training data”}$$

$$\text{if } y_i = f(x_i; \mathbf{w}) + \varepsilon \text{ where } \varepsilon \sim \mathcal{N}(0, \sigma)$$

$$\text{then likelihood would be: } \log L(w) = - \sum_i \frac{(y_i - f(x_i; w))^2}{2\sigma^2} + \text{const}$$

a prior on $\mathbf{w} \sim \mathcal{N}(0, \sigma_w)$ = weight decay or L2 regularization

a prior on $\mathbf{w} \sim \mathcal{N}(0, \sigma_w)$ = weight decay or L2 regularization

$$\text{log-likelihood of NN: } \log L(\mathbf{w}) = - \sum_i \frac{(y_i - f(x_i; \mathbf{w}))^2}{2\sigma^2} + \text{const}$$

a prior on $\mathbf{w} \sim \mathcal{N}(0, \sigma_w)$ = weight decay or L2 regularization

$$\text{log-likelihood of NN: } \log L(\mathbf{w}) = - \sum_i \frac{(y_i - f(x_i; \mathbf{w}))^2}{2\sigma^2} + \text{const}$$

$$p(\mathbf{w}) = \prod_{k=1}^D (2\pi\sigma_w^2)^{-1/2} \exp\left(-\frac{w_k^2}{2\sigma_w^2}\right) = (2\pi\sigma_w^2)^{-D/2} \exp\left(-\frac{\|\mathbf{w}\|^2}{2\sigma_w^2}\right)$$

$$\log p(\mathbf{w}) = -\frac{\|\mathbf{w}\|^2}{2\sigma_w^2} - \frac{D}{2} \log (2\pi\sigma_w^2)$$

a prior on $\mathbf{w} \sim \mathcal{N}(0, \sigma_w)$ = weight decay or L2 regularization

log-posterior of NN:

$$\log p(\mathbf{w} | y) = - \sum_i \frac{(y_i - f(x_i; \mathbf{w}))^2}{2\sigma^2} - \frac{\|\mathbf{w}\|^2}{2\sigma_w^2} + \text{const}$$

a prior on $\mathbf{w} \sim \mathcal{N}(0, \sigma_w)$ = weight decay or L2 regularization

log-posterior of NN:

$$\log p(\mathbf{w} | y) = - \sum_i \frac{(y_i - f(x_i; \mathbf{w}))^2}{2\sigma^2} - \boxed{\frac{\|\mathbf{w}\|^2}{2\sigma_w^2}} + \text{const}$$

$\lambda \sum_i w_i^2$ *L2 regularization*

a prior on $\mathbf{w} \sim \mathcal{N}(0, \sigma_w)$ = weight decay or L2 regularization

log-posterior of NN:

$$\log p(\mathbf{w} | y) = - \sum_i \frac{(y_i - f(x_i; \mathbf{w}))^2}{2\sigma^2} - \boxed{\frac{\|\mathbf{w}\|^2}{2\sigma_w^2}} + \text{const}$$

$\lambda \sum_i w_i^2$ *L2 regularization*

or weight decay via optimizer: `torch.optim.Adam(..., weight_decay= $\lambda$ )`

a prior on $\mathbf{w} \sim \mathcal{N}(0, \sigma_w)$ = weight decay or L2 regularization
reduces overfitting

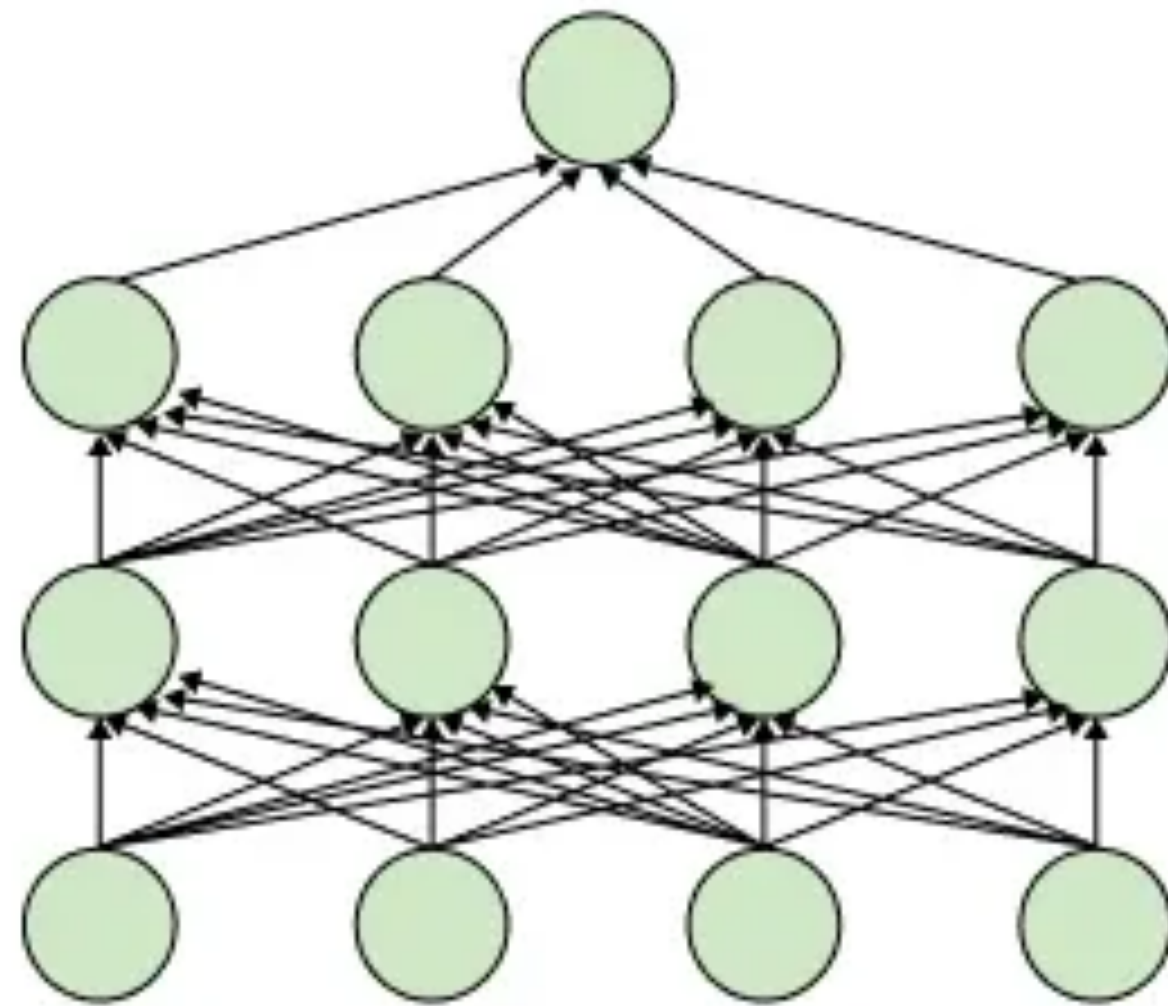
a prior on $\mathbf{w} \sim \mathcal{N}(0, \sigma_w)$ = weight decay or L2 regularization
reduces overfitting

early stopping is also a prior

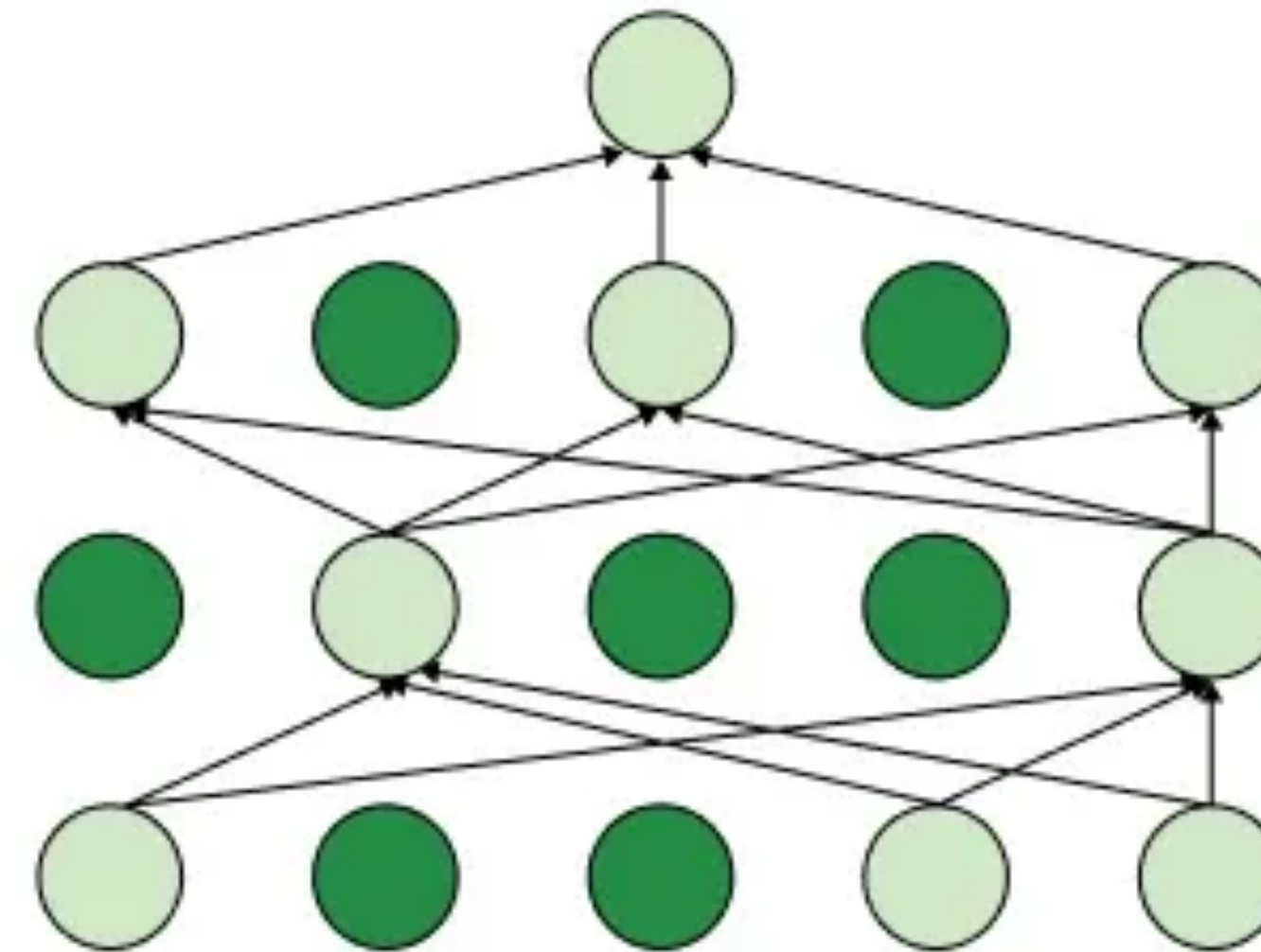
starting with small weights, they can only change
so much after t gradient steps

dropout — set hidden unit to 0 with probability p during training, use every unit at test time

A neural network before dropout



A neural network after dropout



dropout — set hidden unit to 0 with probability p during training, use every unit at test time

no unit can rely on any particular other unit being there, so the network can't build fragile, co-adapted features that only fit the training set

dropout — set hidden unit to 0 with probability p during training, use every unit at test time

no unit can rely on any particular other unit being there, so the network can't build fragile, co-adapted features that only fit the training set

keeping dropout on at test time and averaging many passes gives an approximate posterior predictive (*Gal & Ghahramani 2016*)

dropout — set hidden unit to 0 with probability p during training, use every unit at test time

no unit can rely on any particular other unit being there, so the network can't build fragile, co-adapted features that only fit the training set

keeping dropout on at test time and averaging many passes gives an approximate posterior predictive (*Gal & Ghahramani 2016*)

pytorch: `nn.Dropout(p)`

what are some other prior knowledge we can use?

prior knowledge of the inputs *e.g.* galaxy images

data augmentation

prior knowledge of the inputs *e.g.* galaxy images

data augmentation

augment the training data by randomly rotating and flipping galaxy
images and keeping the copies

tells the network the output is **invariant under these transformations**

prior on the function/neural network (or inductive bias)

prior knowledge of the inputs *e.g.* galaxy images

data augmentation

augment the training data by randomly rotating and flipping galaxy images and keeping the copies

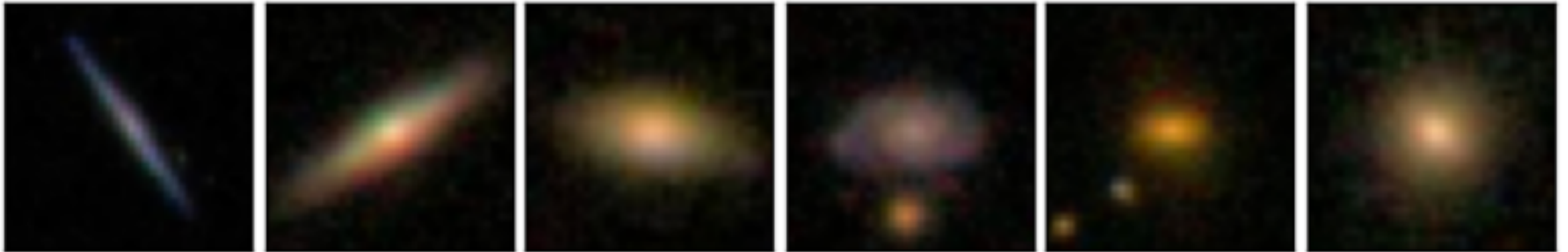
tells the network the output is **invariant under these transformations**

prior on the function/neural network (or inductive bias)

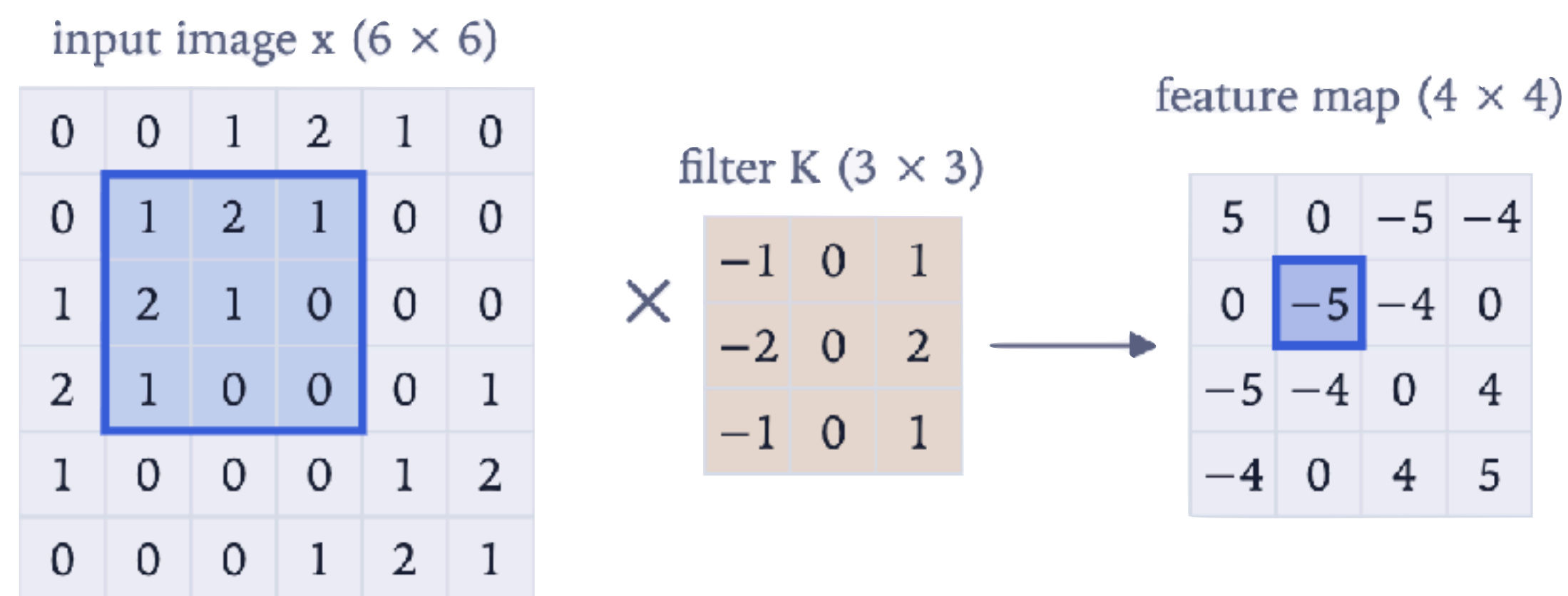
can we impose more explicit function priors?

architectures

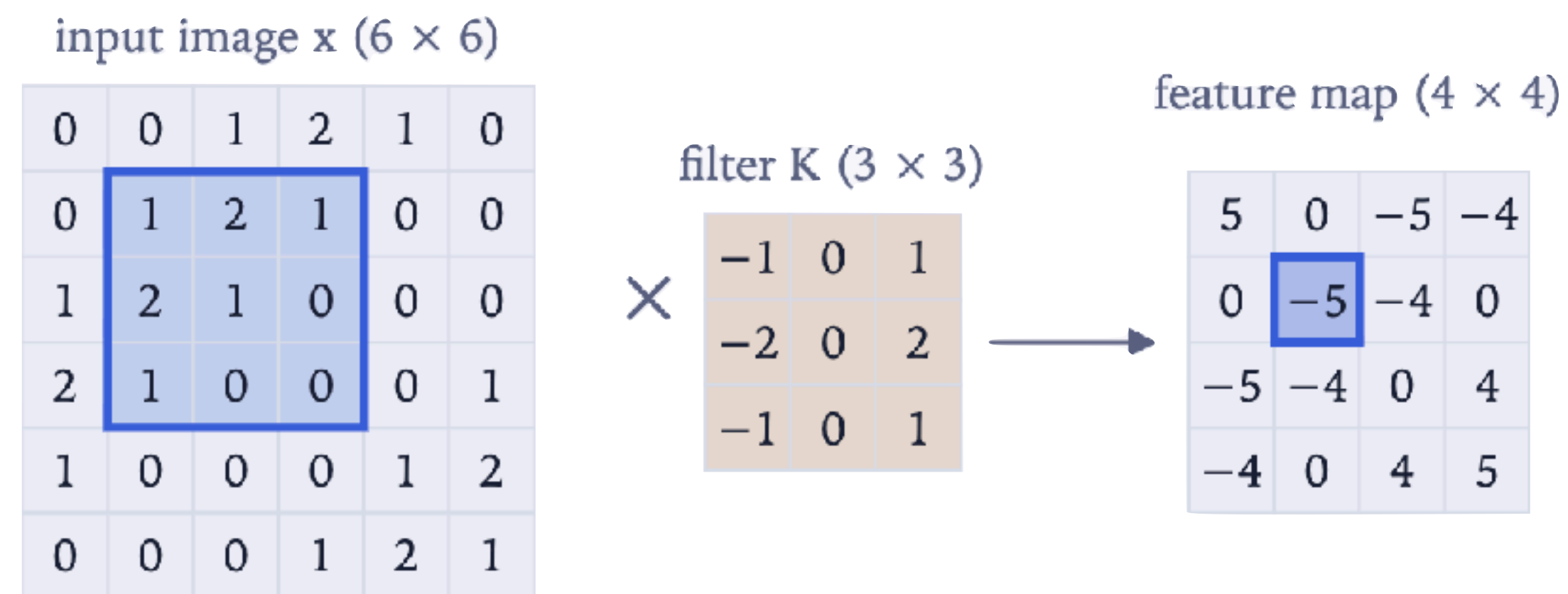
images



convolutional neural networks: a filter slid across the image

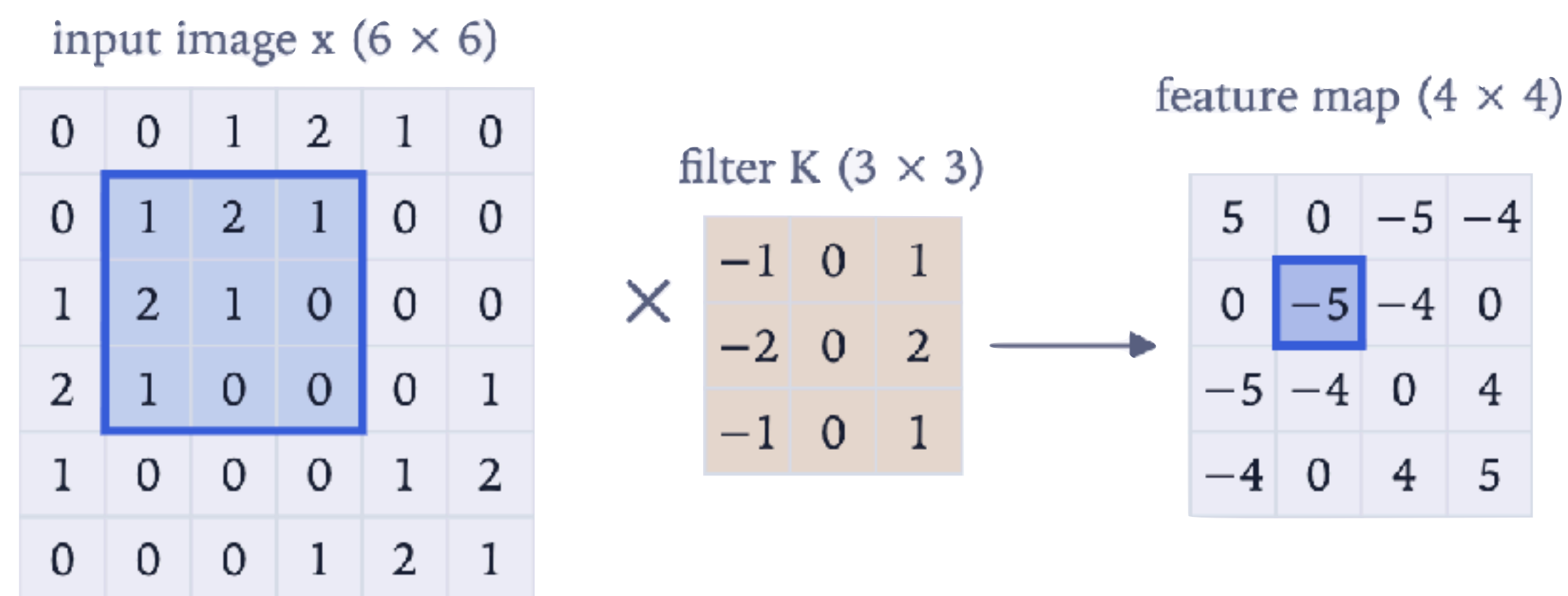


convolutional neural networks: a filter slid across the image



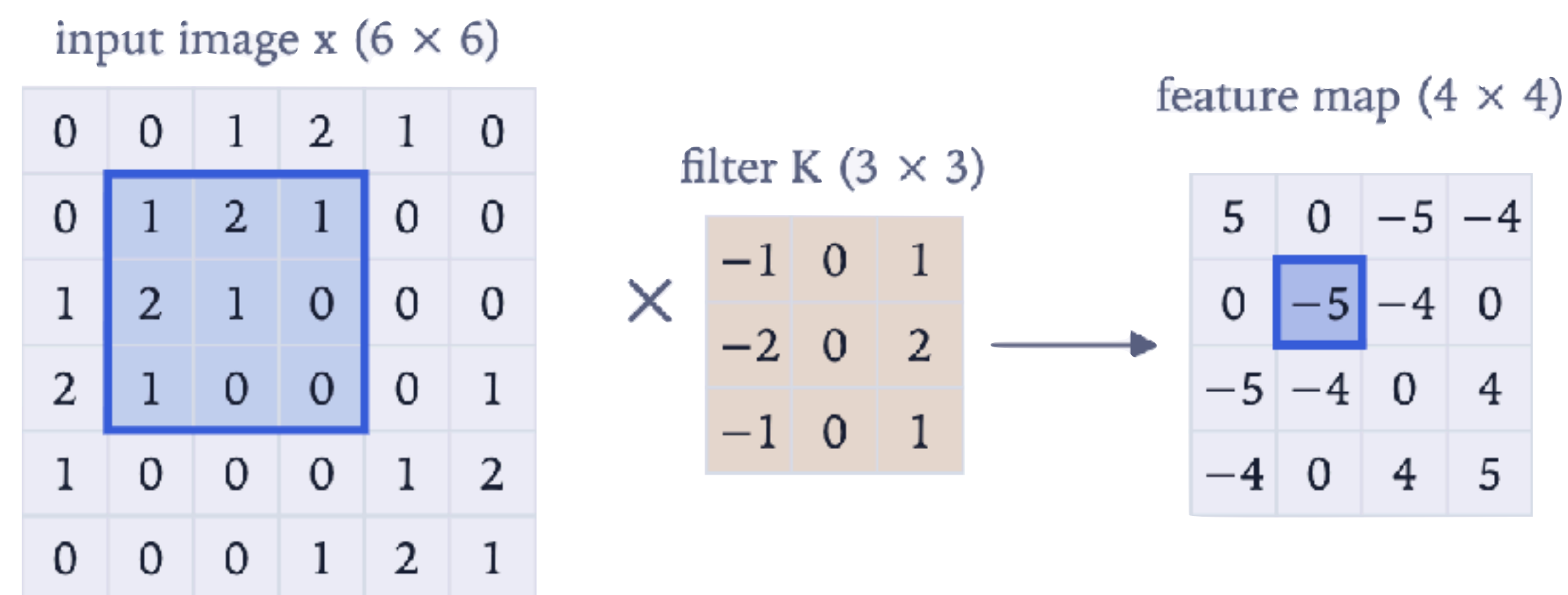
$$\begin{aligned} &(-1) \cdot 1 + 0 \cdot 2 + 1 \cdot 1 = 0 \\ &(-2) \cdot 2 + 0 \cdot 1 + 2 \cdot 0 = -4 \\ &(-1) \cdot 1 + 0 \cdot 0 + 1 \cdot 0 = -1 \\ &\quad \quad \quad = -5 \end{aligned}$$

convolutional neural networks: a filter slid across the image



$$\begin{aligned}(-1) \cdot 1 + 0 \cdot 2 + 1 \cdot 1 &= 0 \\(-2) \cdot 2 + 0 \cdot 1 + 2 \cdot 0 &= -4 \\(-1) \cdot 1 + 0 \cdot 0 + 1 \cdot 0 &= -1 \\&= -5\end{aligned}$$

convolutional neural networks: a filter slid across the image



$$h_{ij} = \sigma \left(\sum_{p,q} K_{p,q} x_{i+p,j+q} + b \right)$$

same filter K at every position

convolutional neural networks: a filter slid across the image

input image x (6×6)

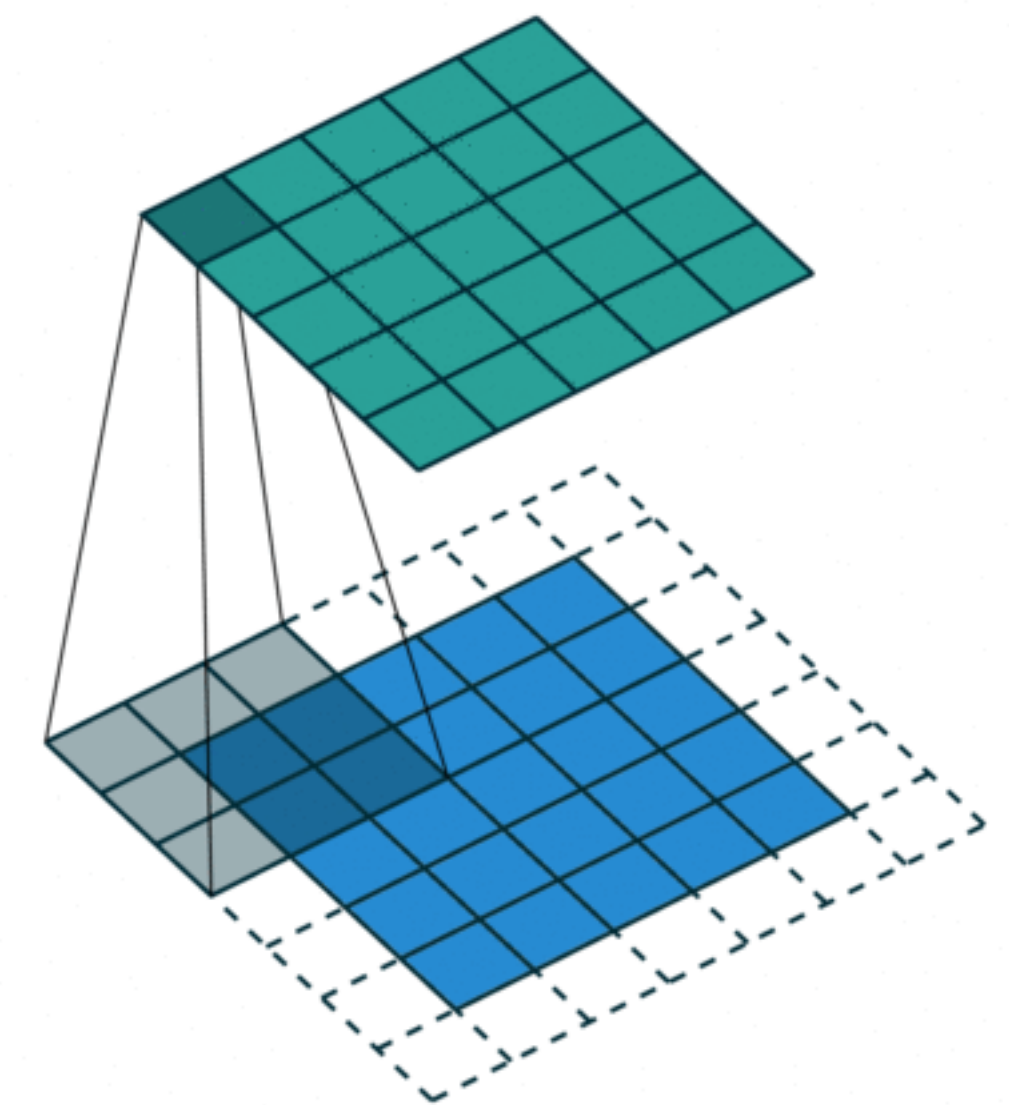
0	0	1	2	1	0
0	1	2	1	0	0
1	2	1	0	0	0
2	1	0	0	0	1
1	0	0	0	1	2
0	0	0	1	2	1

filter K (3×3)

$\times$	-1	0	1
	-2	0	2
	-1	0	1

feature map (4×4)

5	0	-5	-4
0	-5	-4	0
-5	-4	0	4
-4	0	4	5



$6 \times 6 \rightarrow 4 \times 4$ (without padding)

$6 \times 6 \rightarrow 6 \times 6$ (with padding)

convolutional neural networks: pooling

input image x (6 × 6)

0	0	1	2	1	0
0	1	2	1	0	0
1	2	1	0	0	0
2	1	0	0	0	1
1	0	0	0	1	2
0	0	0	1	2	1

filter K (3 × 3)

×

-1	0	1
-2	0	2
-1	0	1

feature map (4 × 4)

5	0	-5	-4
0	-5	-4	0
-5	-4	0	4
-4	0	4	5

ReLU

5	0	0	0
0	0	0	0
0	0	0	4
0	0	4	5

is the pattern here at all?

max-pool 2 × 2

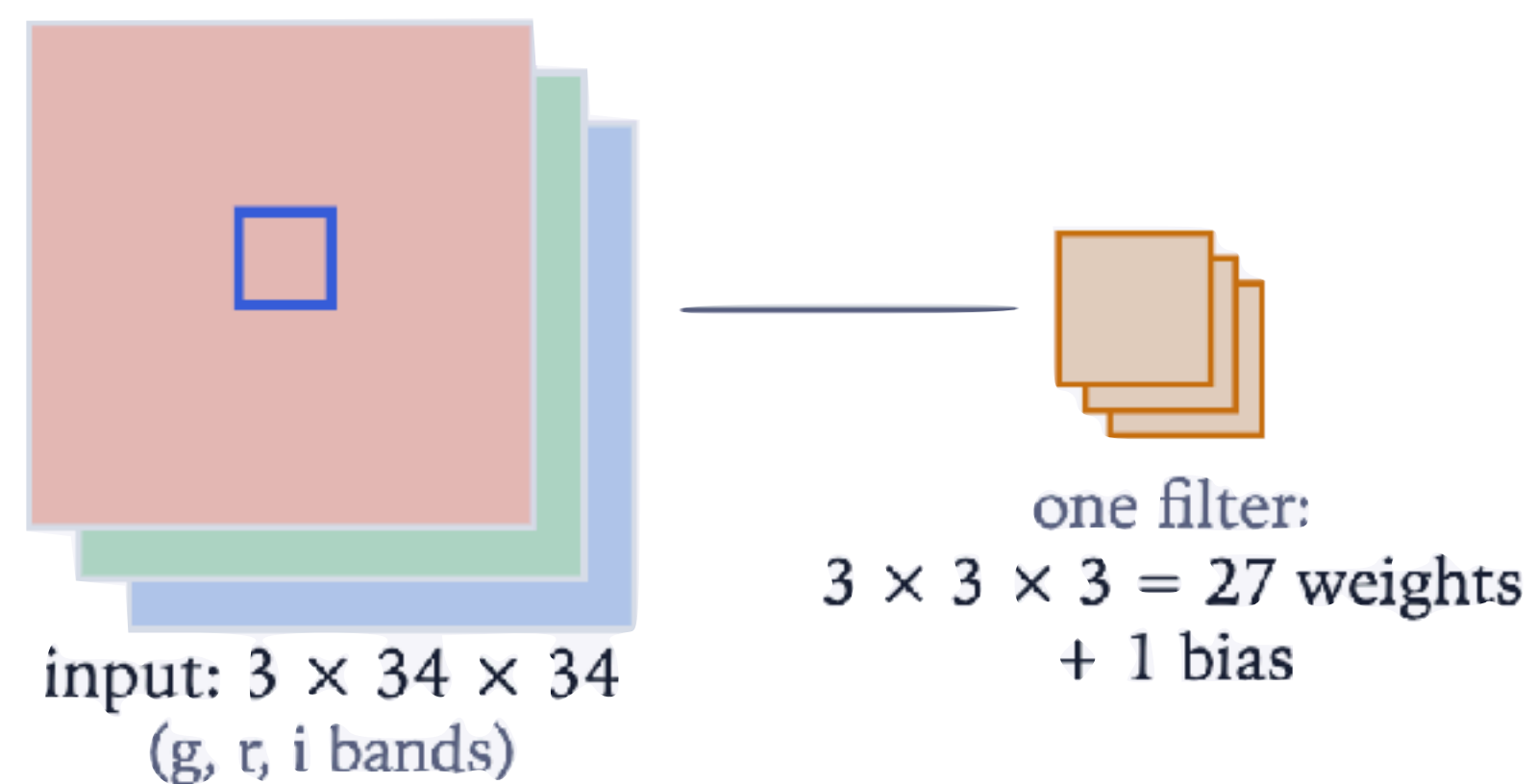
5	0
0	5

average-pool 2 × 2

1.25	0
0	3.25

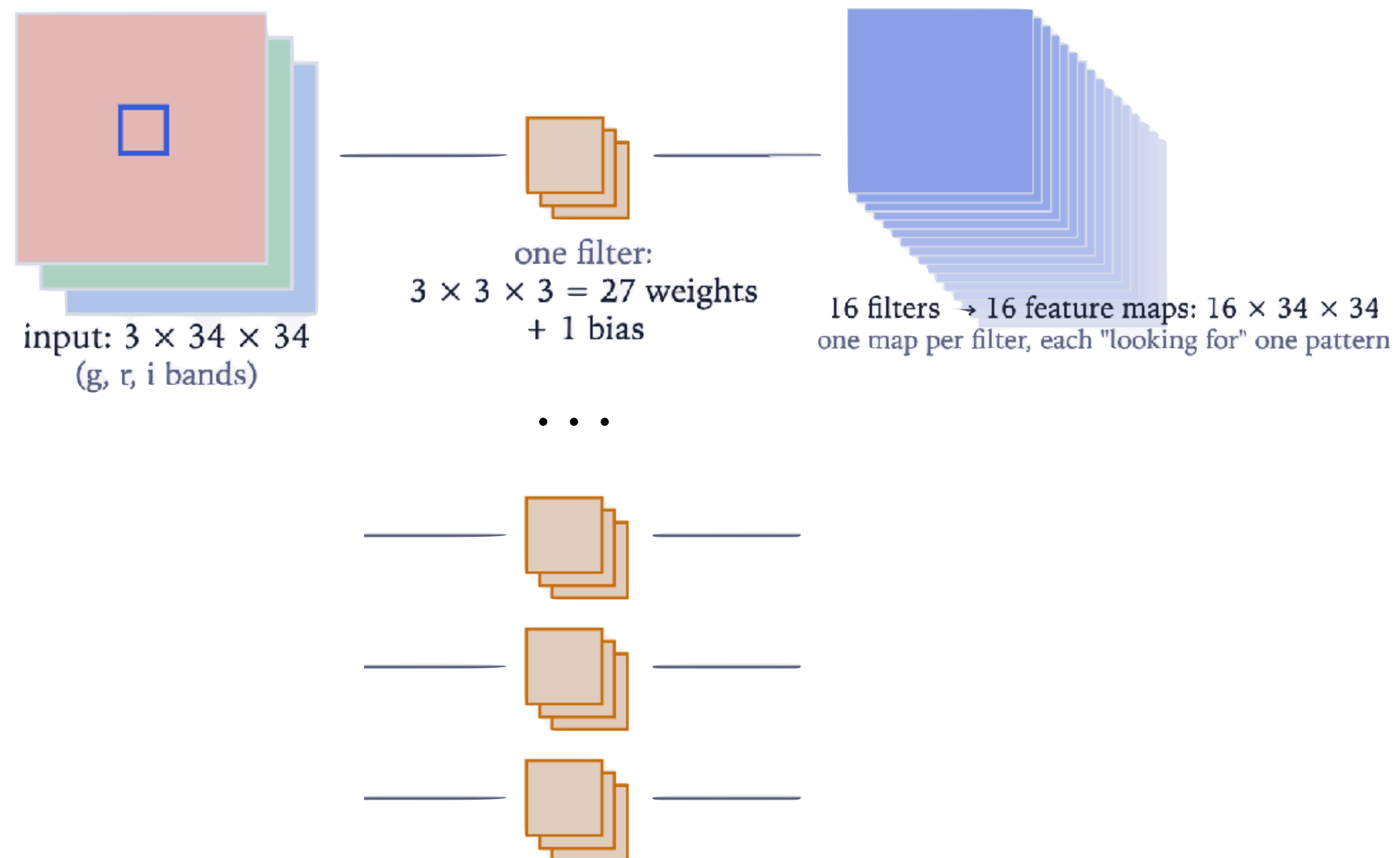
how much of it on average?

convolutional neural networks: multiple channels

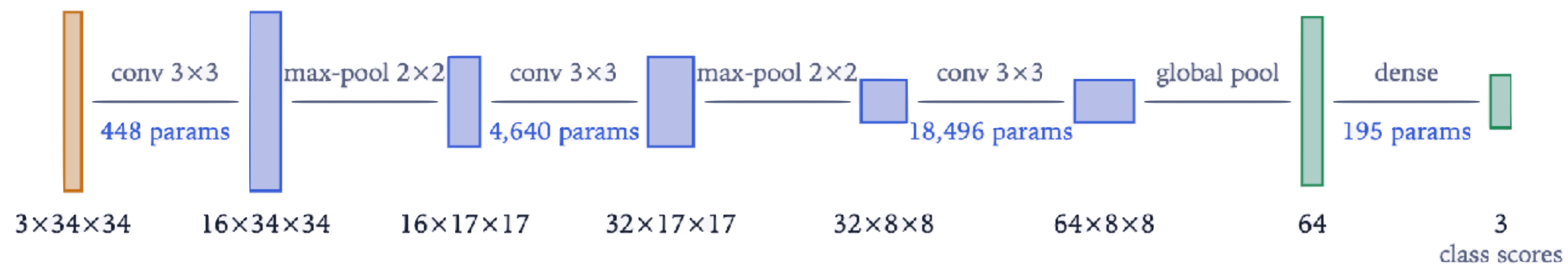


$$h_{ij} = \sigma \left(\sum_{p,q,c} K_{p,q,c} x_{i+p,j+q,c} + b \right)$$

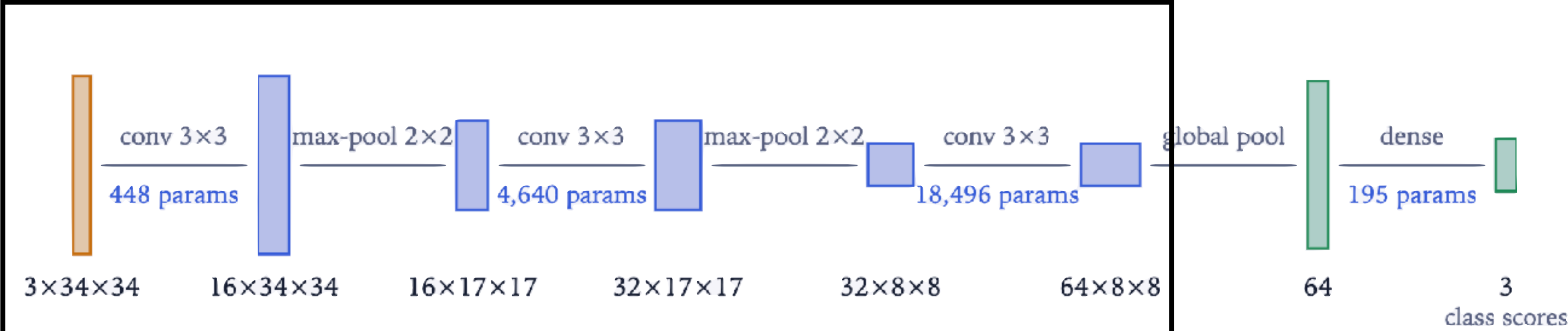
convolutional neural networks: multiple channels, many filters



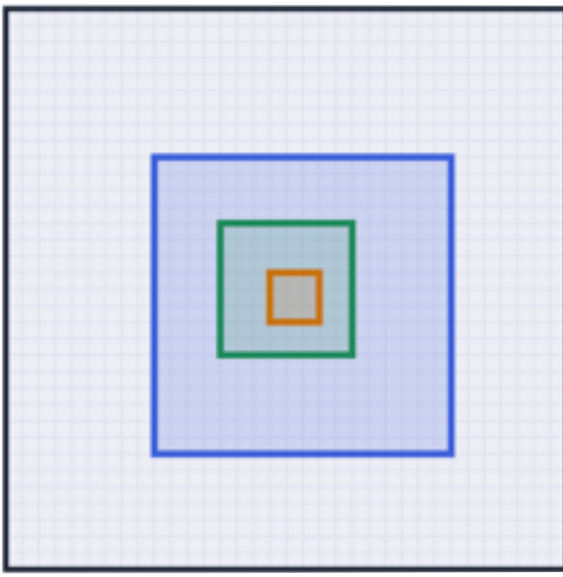
convolutional neural networks



convolutional neural networks

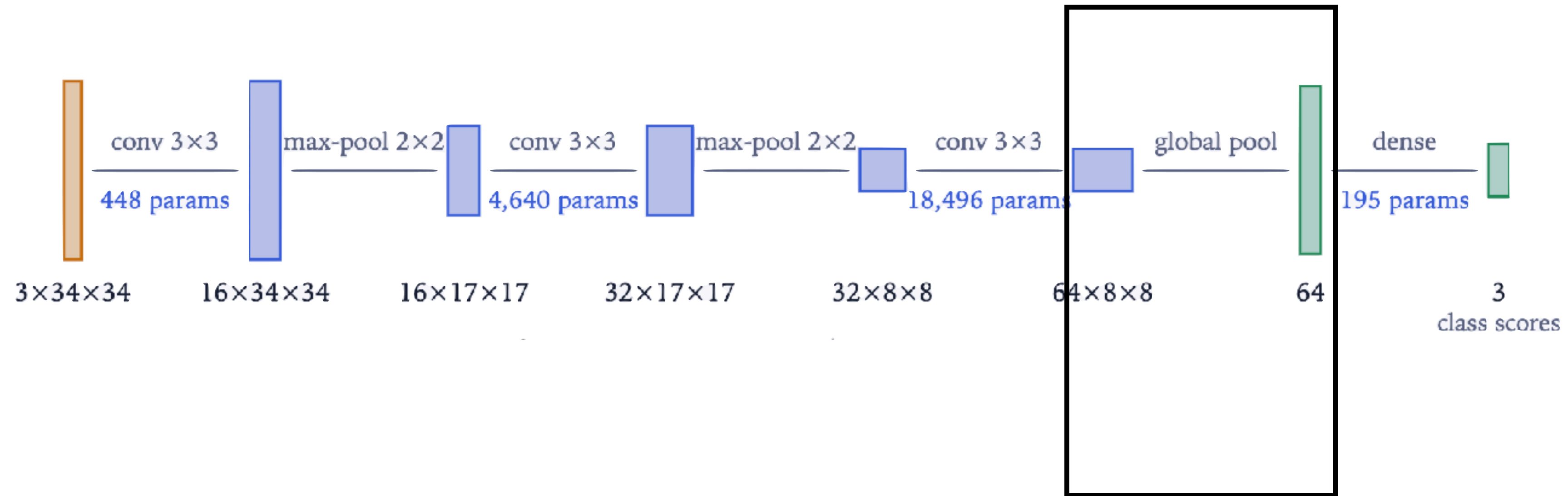


extracting features on different scales



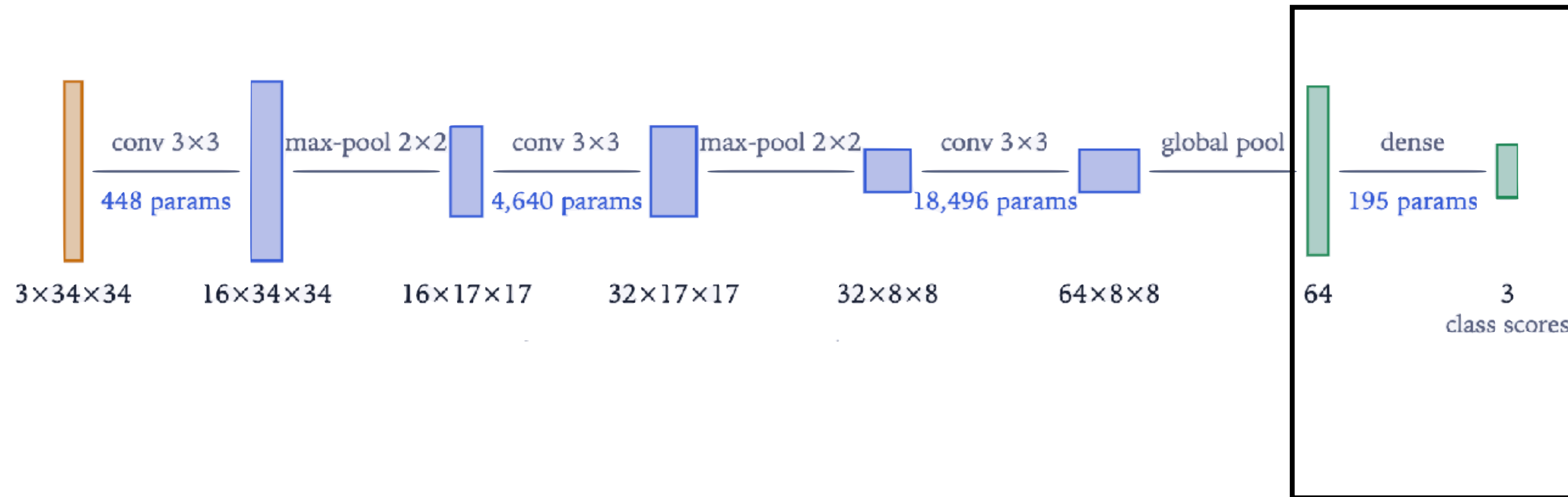
- after conv 1: 3 px = 2.4"
 - after conv 2: 8 px = 6.3"
 - after conv 3: 18 px = 14.3"
 - after global average pool: all 34 px = 27"
- one unit's view of the 34×34 image (0.79" per pixel)

convolutional neural networks



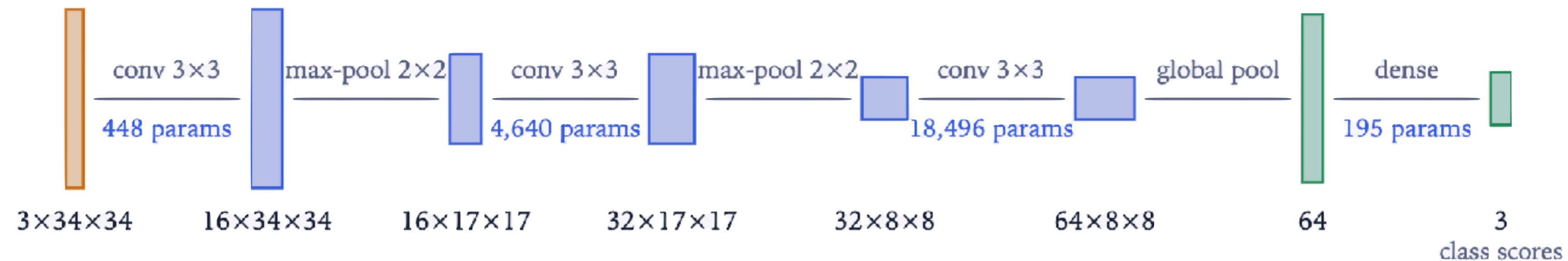
one average per image
position no longer matters

convolutional neural networks



maps the CNN output to
inference or classification

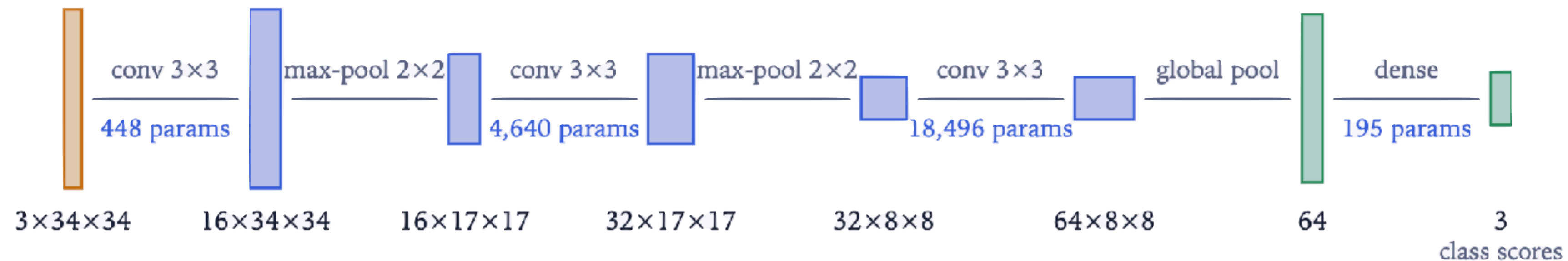
convolutional neural networks



equivariant: shift the galaxy, and the feature maps shift with it

invariant: shift the galaxy, and the answer doesn't change

convolutional neural networks

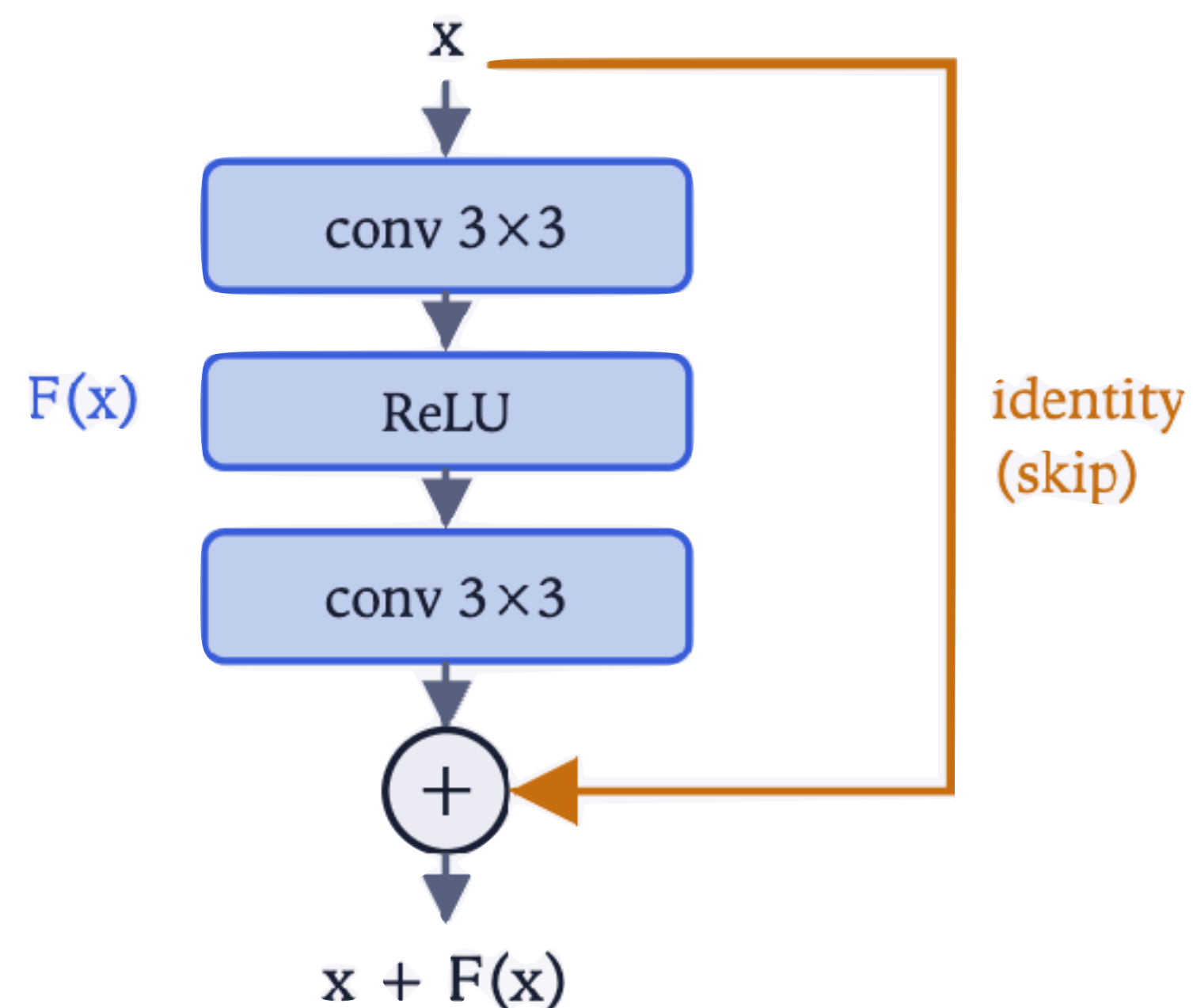


equivariant: shift the galaxy, and the feature maps shift with it

invariant: shift the galaxy, and the answer doesn't change

translation but not rotation

deep convolutional neural networks: *ResNet* and *U-Nets*

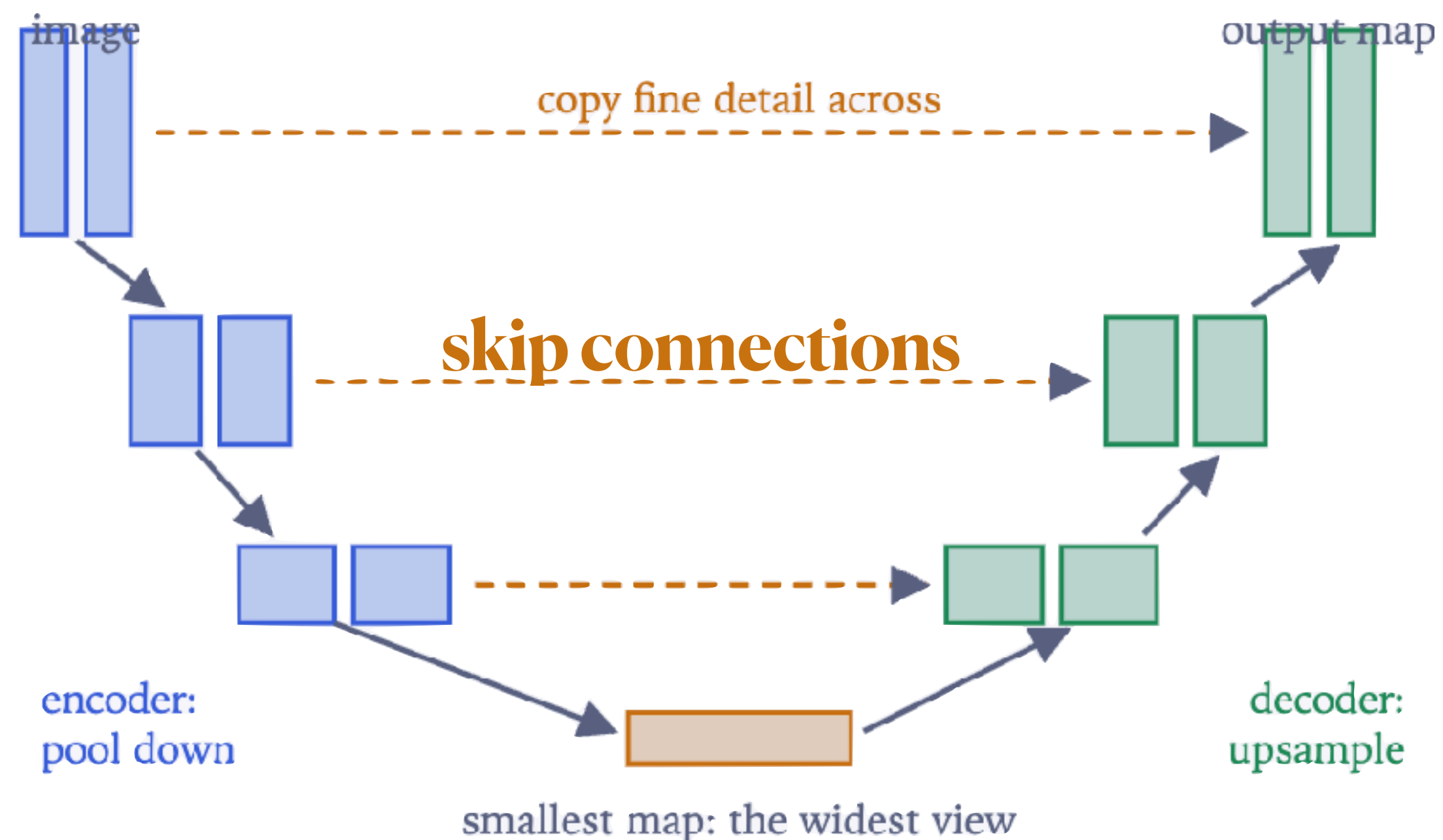


ResNet (*He et al. 2016*): each block learns only a correction $F(x)$

$$\frac{\partial(x + F)}{\partial x} = 1 + \frac{\partial F}{\partial x}$$

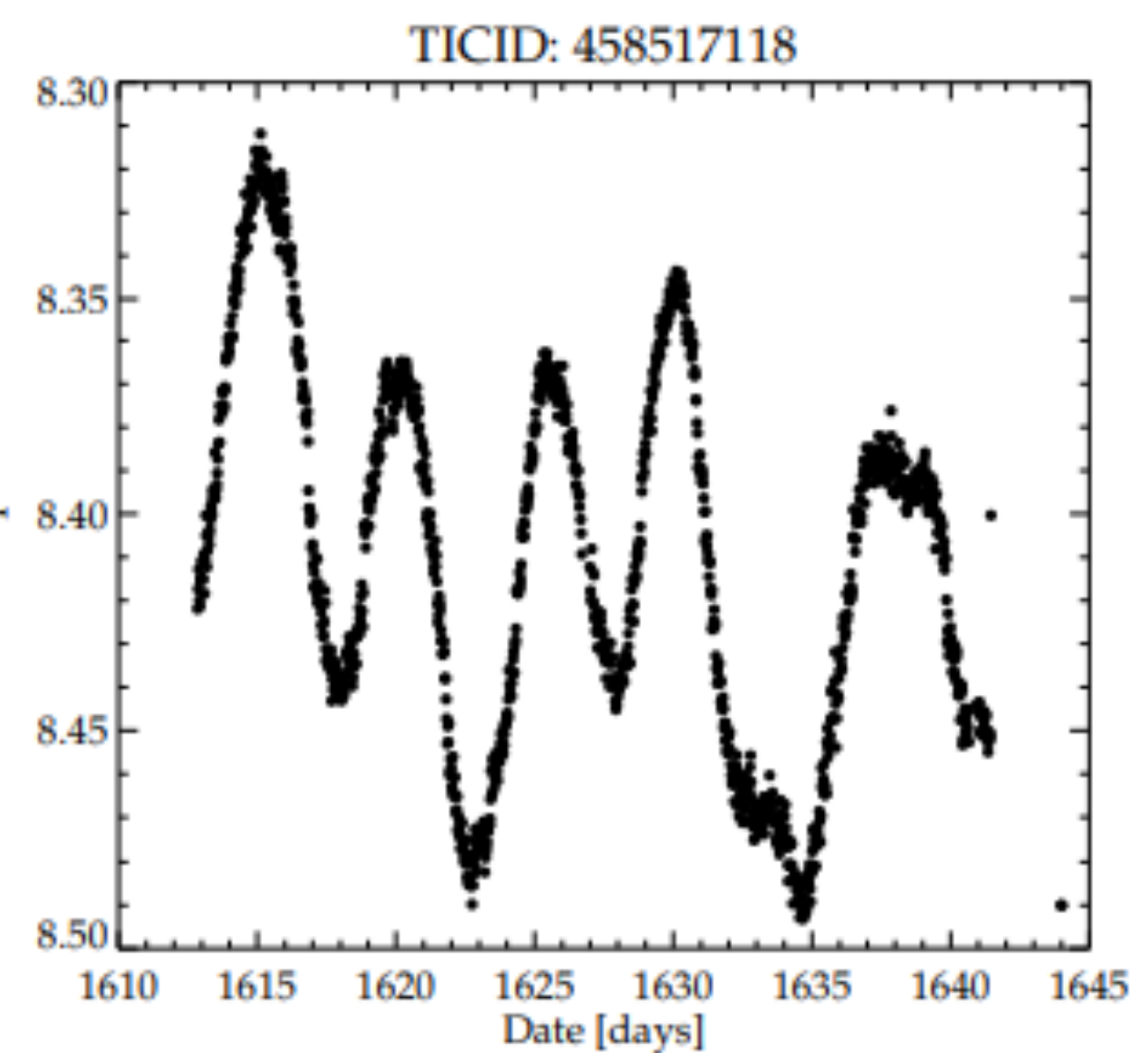
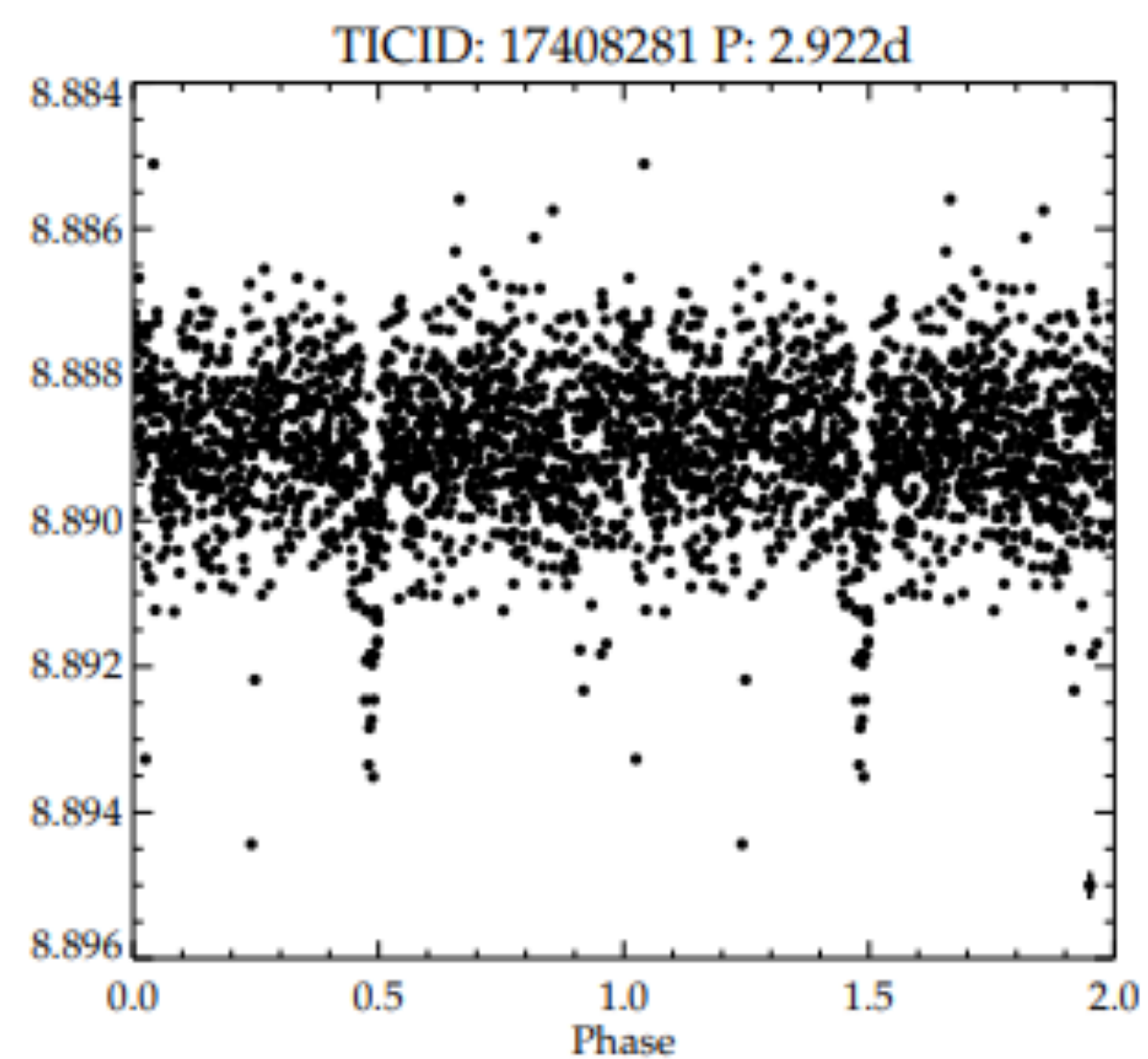
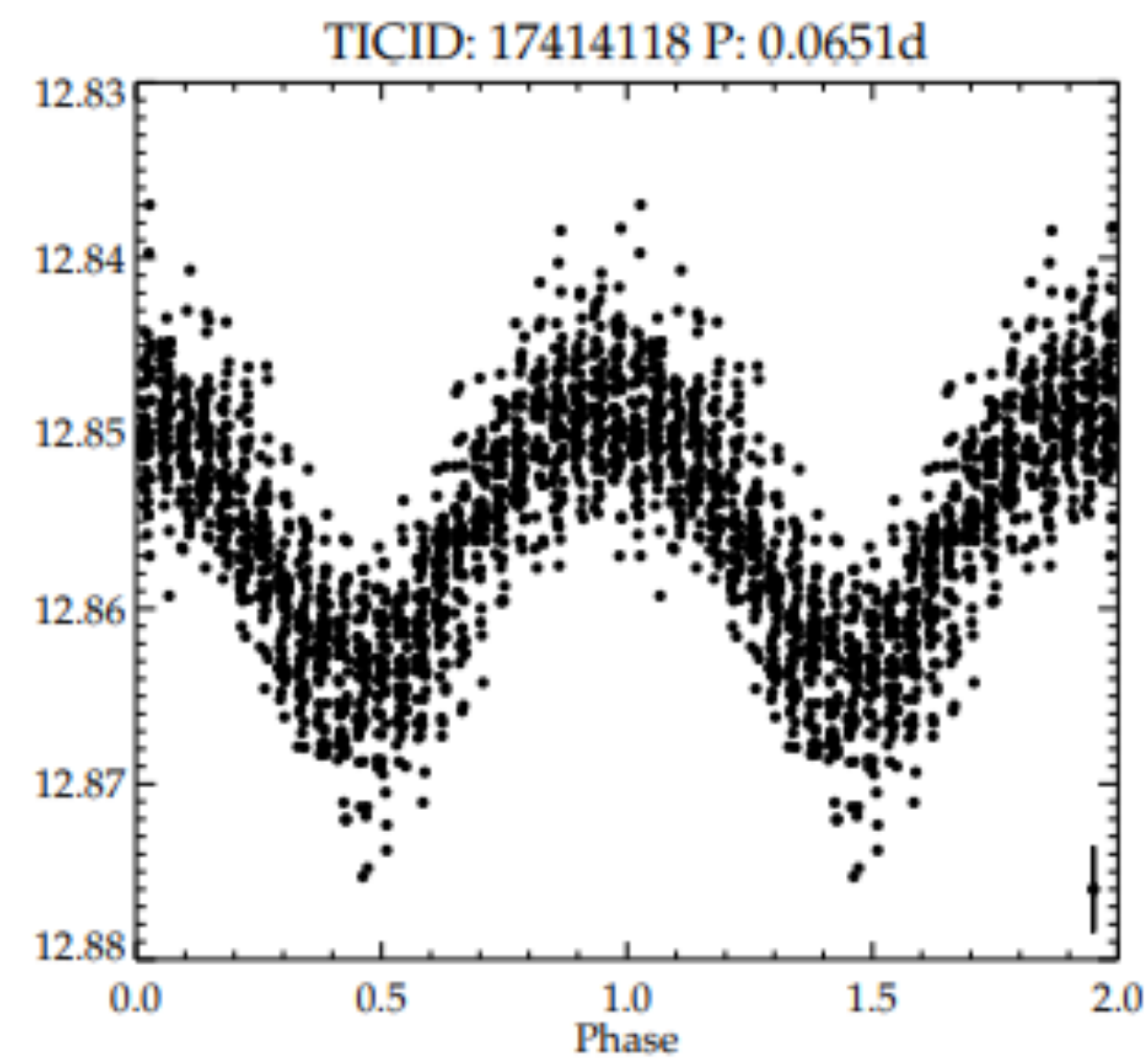
you can make networks much deeper by solving the degradation problem (*more layers degrades accuracy due to optimization*)

deep convolutional neural networks: *ResNet* and *U-Nets*



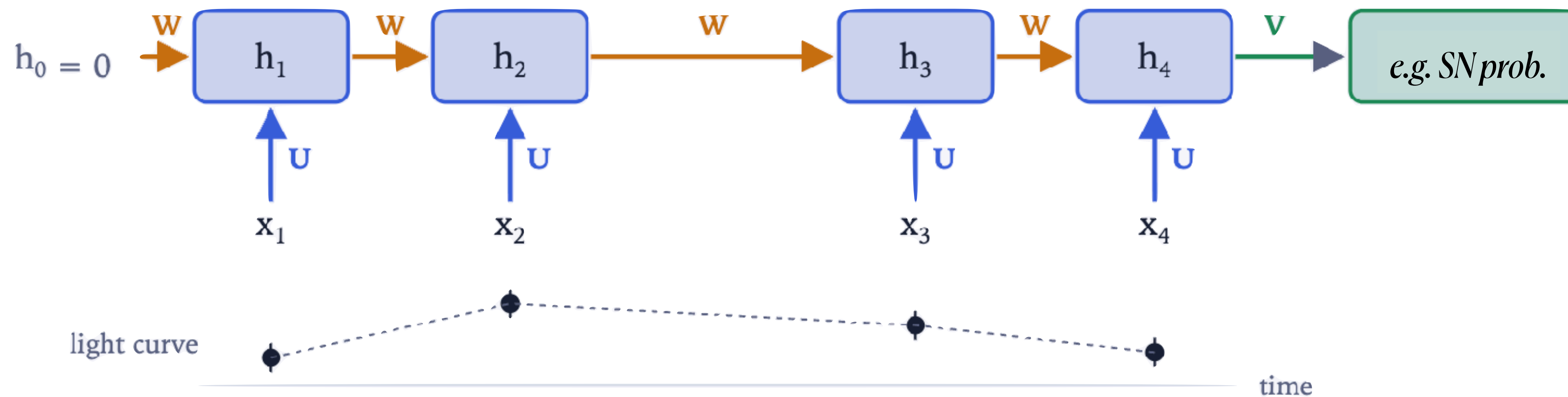
U-Net (*Ronneberger et al. 2015*): skip connections pass the encoder maps at each level to restore the detail pooling threw away.

sequenced data (light curves)



recurrent networks for ordered sequences (*light curve*)

$$h_t = \sigma(W h_{t-1} + U x_t + b)$$

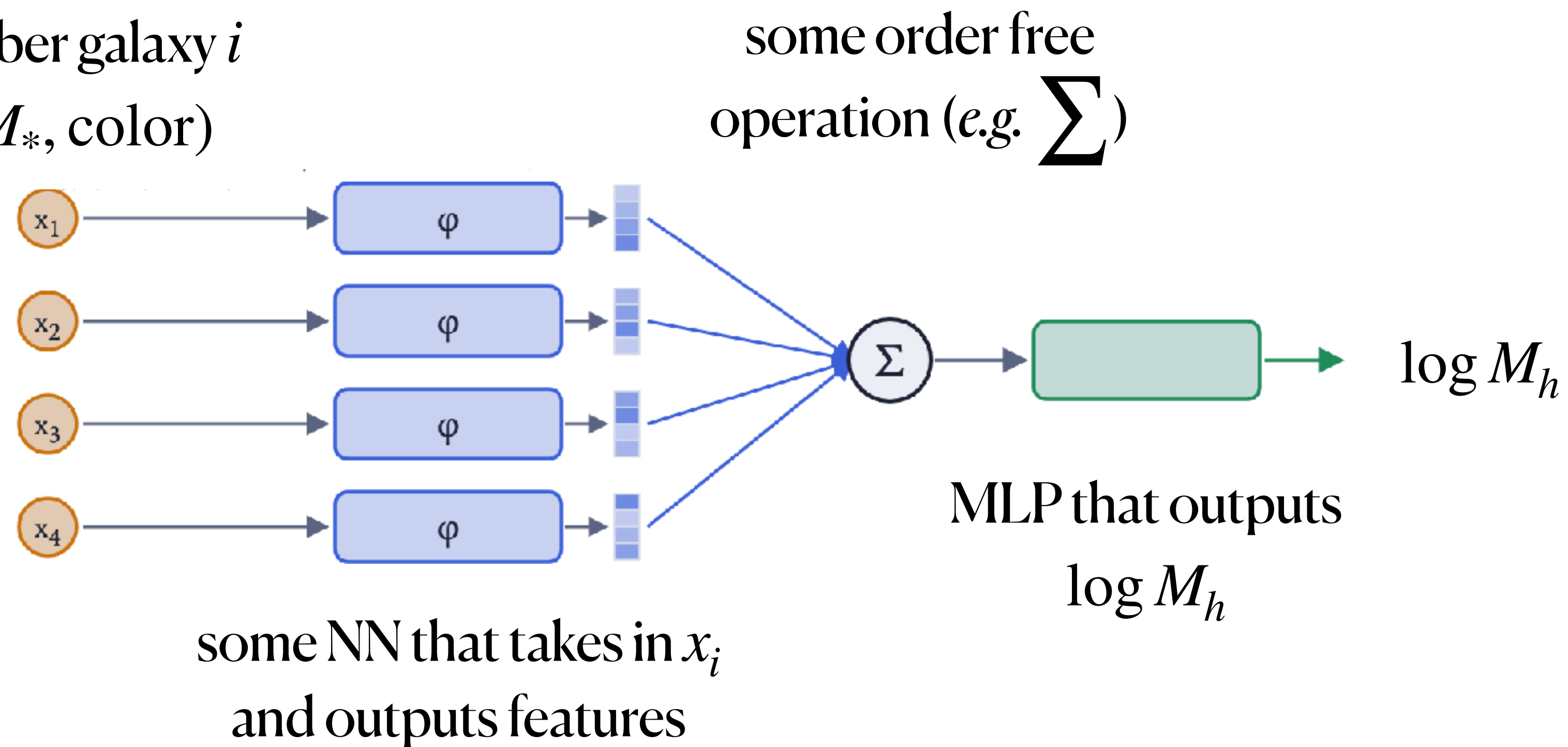


$$e.g. x_t = (t, flux, flux\ error)$$

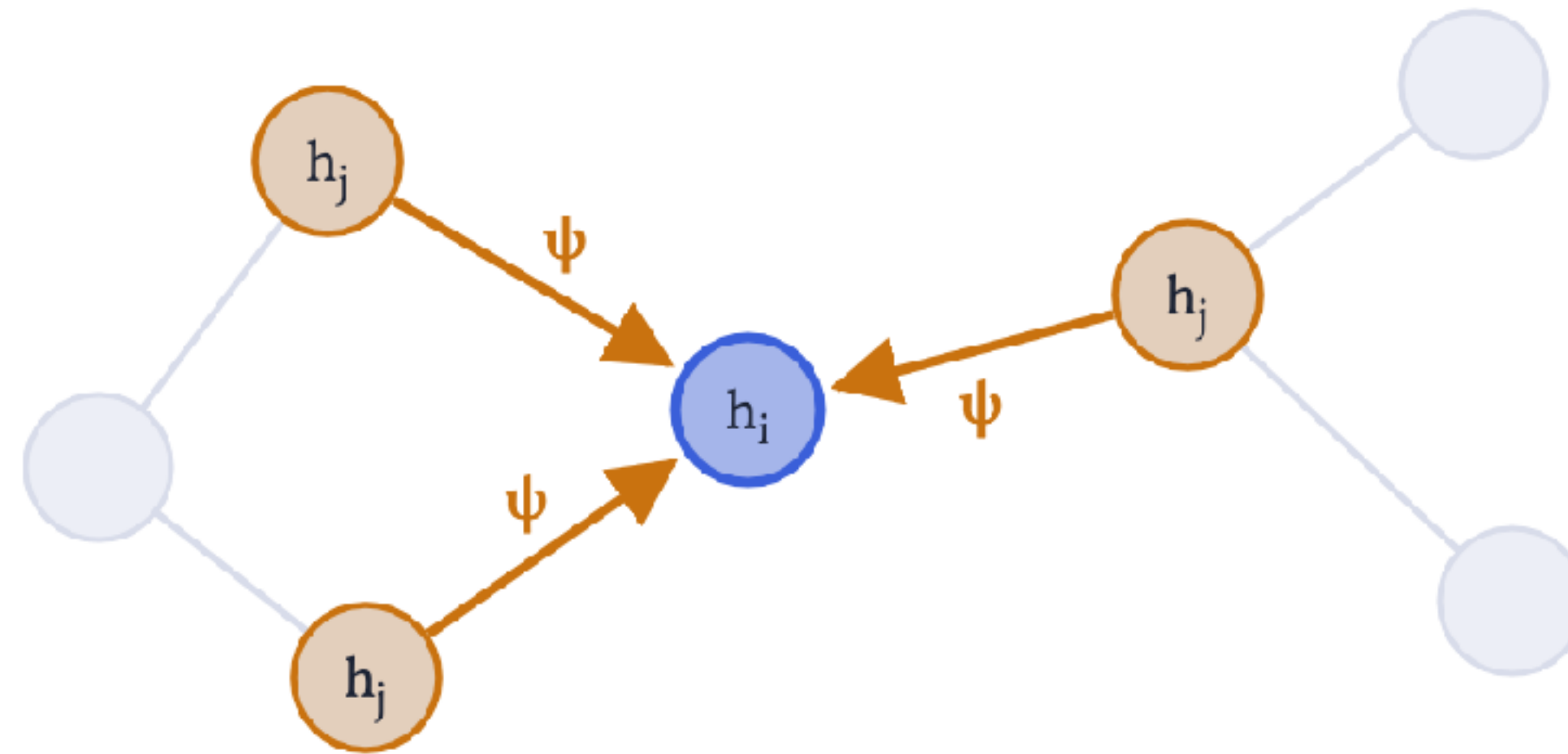
deep sets (*Zaheer et al. 2017*) for unordered data:

e.g. cluster halo mass given properties of member galaxies

e.g. member galaxy i
 $x_i = (M_*, \text{color})$

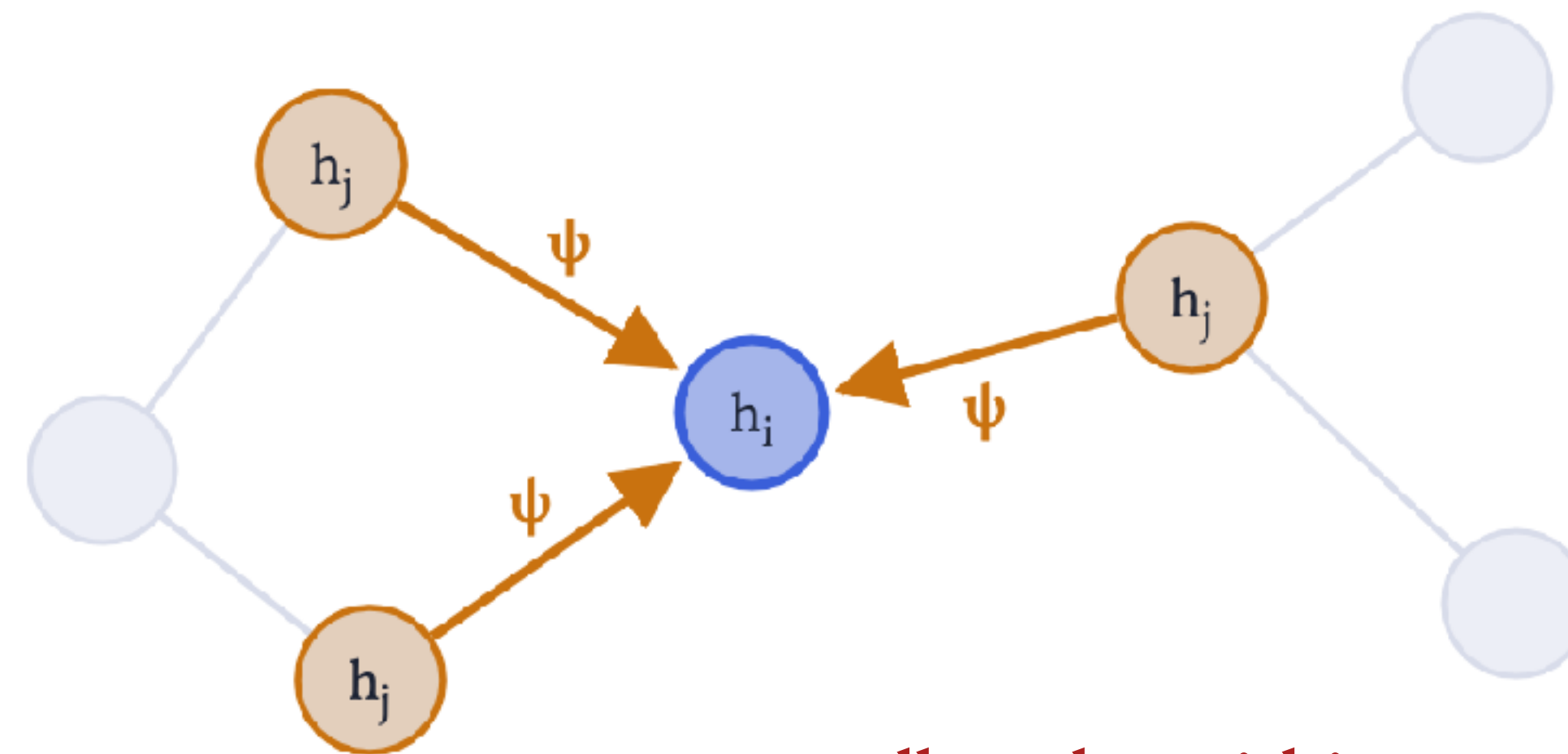


graph neural networks



update node feature $h_i^{\ell+1} = \phi^{\ell} \left(h_i^{\ell}, \sum_{j \in N(i)} \psi^{\ell}(h_i^{\ell}, h_j^{\ell}, e_{i,j}) \right)$

graph neural networks



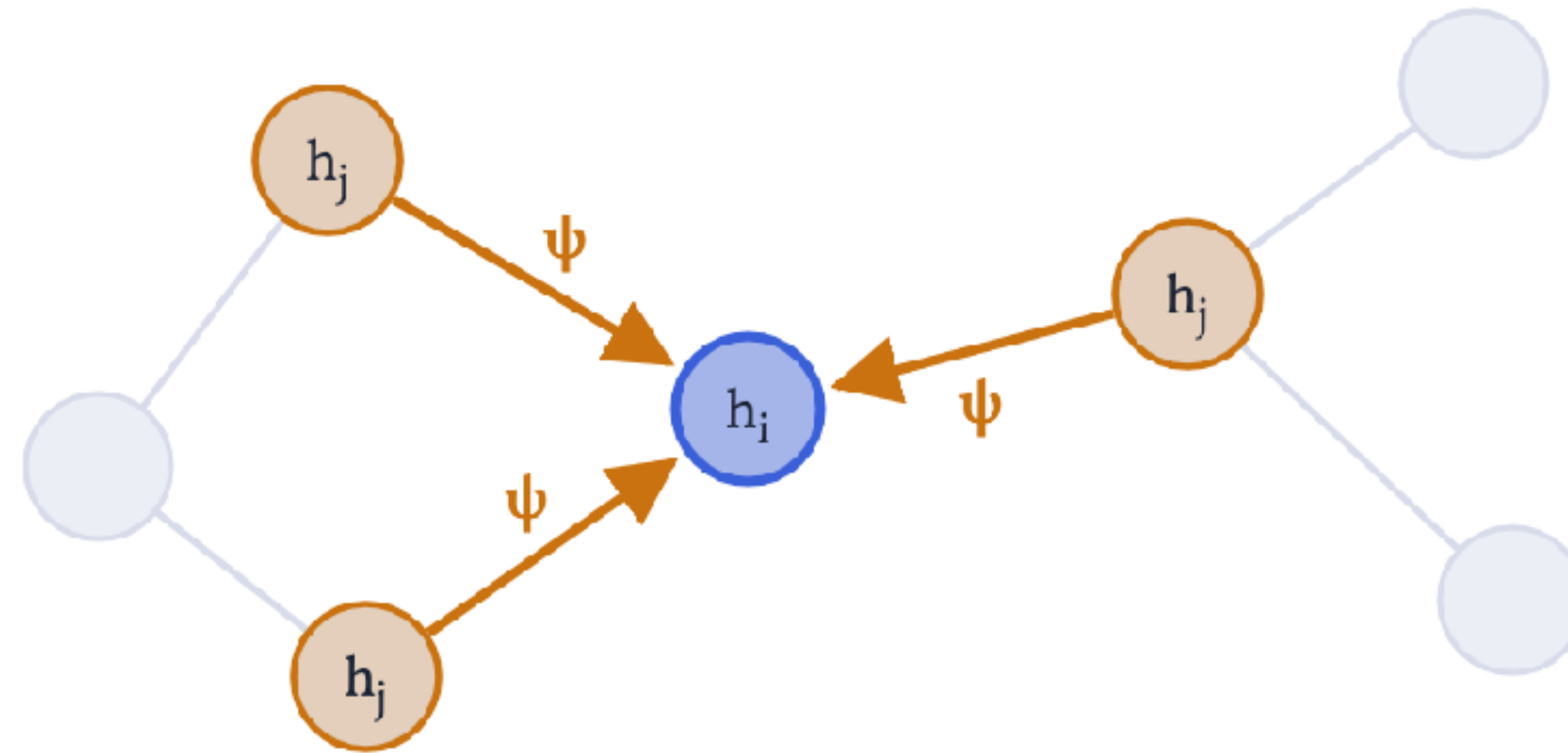
aggregate all nodes within some linking length

update node feature

$$h_i^{\ell+1} = \phi^{\ell} \left(h_i^{\ell}, \sum_{j \in N(i)} \psi^{\ell}(h_i^{\ell}, h_j^{\ell}, e_{i,j}) \right)$$

a “message” along each edge, based on node
and edge features h and e

graph neural networks

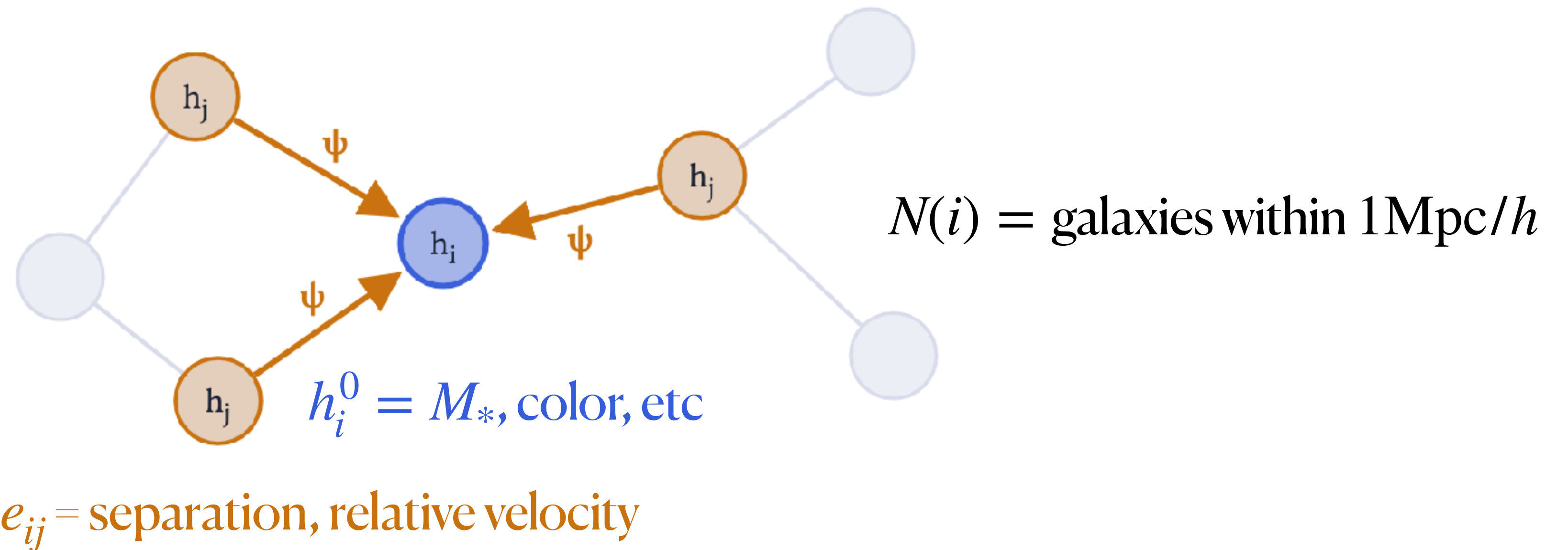


after L rounds

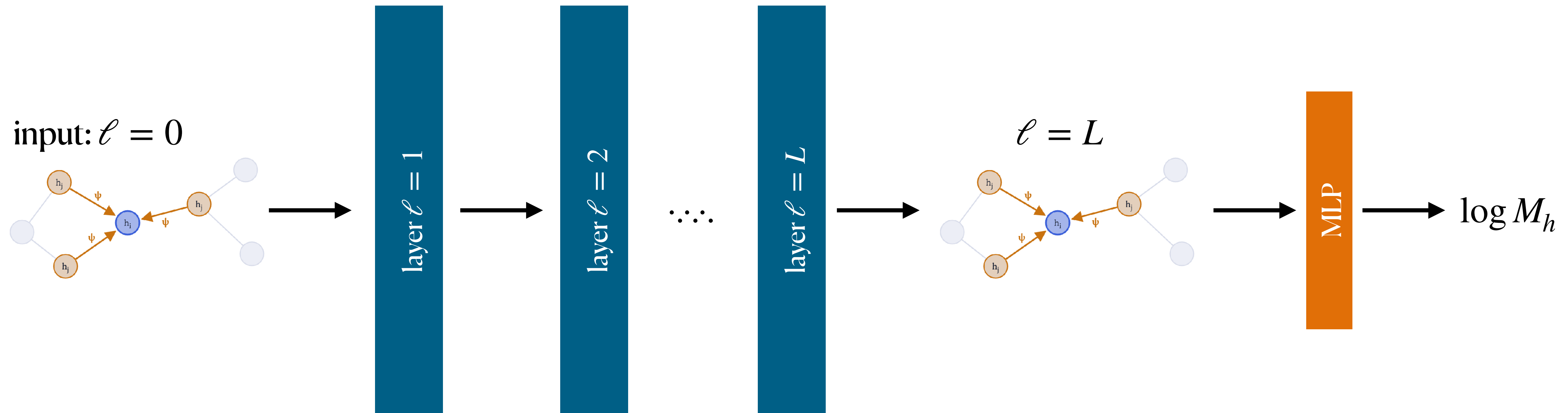
$$\rho \left(\sum_i h_i^L \right) \rightarrow \text{output}$$

graph neural networks: *e.g. cluster halo mass estimation*

input: $\ell = 0$



graph neural networks: *e.g. cluster halo mass estimation*



match the architecture to the symmetry of your data!

architecture	built-in symmetry	
MLP	none	
CNN	translation	<i>images</i>
RNN	shift in time	<i>light curves</i>
Deepset/GNN	permutation	<i>point clouds</i>

match the architecture to the symmetry of your data!

architecture	built-in symmetry	
MLP	none	
CNN	translation	<i>images</i>
RNN	shift in time	<i>light curves</i>
Deepset/GNN	permutation	<i>point clouds</i>

with enough data you can often learn the structure anyway

symmetry is only approximate

some architectures are slower and more expensive

match the architecture to the symmetry of your data!

architecture	built-in symmetry	
MLP	none	
CNN	translation	<i>images</i>
RNN	shift in time	<i>light curves</i>
Deepset/GNN	permutation	<i>point clouds</i>

think of it like a *prior*: biggest bang for the buck when the symmetry is correct and data are scarce

in-class: galaxy morphology with a convolutional neural network

ast384c/inclass/lec13_cnn/galaxy_cnn.ipynb